SE/COM S 3190 - Construction of User Interfaces
Final Project Technical Report

Spring 2026

Saint Cy's Hospital

Team Members

Devank Uppal duppal@iastate.edu

Neal Sharma nea1@iastate.edu

Iowa State University
Department of Computer Science

Contents

1 Introduction **3**
- 1.1 Project Overview . . . 3
- 1.2 Target Users and Use Cases . . . 3
- 1.3 Core Features . . . 4
- 1.4 Originality vs Inspiration . . . 4

2 System Overview and Project Description **5**
- 2.1 Major Functional Modules . . . 5
- 2.2 UI and Navigation Flow . . . 5
- 2.3 CRUD Entities . . . 6

3 File and Folder Architecture **7**
- 3.1 Folder Structure Overview . . . 7
- 3.2 Backend Folder Structure . . . 7
  - 3.2.1 Backend Folder Tree . . . 7
  - 3.2.2 Backend Folder Description . . . 7
- 3.3 Frontend Folder Structure . . . 8
  - 3.3.1 Frontend Folder Tree . . . 8
  - 3.3.2 Frontend Folder Description . . . 9

4 Code Explanation and Logic Flow **9**
- 4.1 Frontend–Backend Interaction . . . 9
- 4.2 React Component Example . . . 10
- 4.3 API Route Examples . . . 11
  - 4.3.1 POST Example: Login . . . 11
  - 4.3.2 GET Example: Patient Medications . . . 11
  - 4.3.3 PUT Example: Update Medication . . . 11
  - 4.3.4 DELETE Example: Remove Medication . . . 11
  - 4.3.5 POST Example: AI Assistant . . . 12
- 4.4 Database Interaction . . . 12

5 Screenshots and Web Views **12**
- 5.1 Home Page . . . 13
- 5.2 Login and Signup Page . . . 13
- 5.3 Patient Dashboard . . . 13
- 5.4 Patient Pharmacy View . . . 15
- 5.5 Appointments Page . . . 15
- 5.6 Doctor Portal . . . 16
- 5.7 Security and Settings Page . . . 16

6 Installation and Setup Instructions **18**
- 6.1 Frontend Setup . . . 18
- 6.2 Backend Setup . . . 18
- 6.3 Database Setup . . . 19
- 6.4 Environment Variables . . . 19

7 Contribution Overview **20**
7.1 Devank Uppal Contributions 20
7.2 Neal Sharma Contributions 21
8 Challenges Faced **21**
8.1 Devank Uppal Challenges 21
8.2 Neal Sharma Challenges 22
9 Final Reflections **22**
9.1 Devank Uppal Reflection 22
9.2 Neal Sharma Reflection 22

1 Introduction

1.1 Project Overview

Saint Cy's Hospital is a full-stack hospital management web application built for SE/COM S 3190. The application supports role-based workflows for patients, doctors, and administrators. The core problem addressed by the project is that hospital information is usually split across separate systems: appointment scheduling, patient health details, pharmacy information, prescriptions, user administration, and support tools. Saint Cy's combines these workflows into one web application with a consistent interface.

The application is relevant because healthcare systems need clear access control and organized data flow. A patient should not see administrator tools, a doctor should not manage system users, and each role should only see the tools that match its responsibilities. Saint Cy's uses authentication, role-based routing, MongoDB Atlas, and protected API routes to separate these responsibilities.

At a high level, the system allows users to log in, view pages based on their role, manage appointments, update health/profile data, manage medications, review system errors, and use a Gemini-powered AI assistant for general health guidance. The project scope is a React single-page web application with an Express backend and MongoDB Atlas database.

1.2 Target Users and Use Cases

The system is designed for three main user roles:

- **Patients**: Patients create accounts, log in, view their dashboard, manage health details, book appointments, view notifications, see generic and assigned medications, use a mock pharmacy cart, and ask basic questions through the AI assistant.
- **Doctors**: Doctors manage appointment requests, create available time slots, view patient information connected to appointments, and create prescriptions/medication assignments.
- **Administrators**: Admins manage user accounts, roles, emails, password resets, and system error logs.

Typical use cases include a patient signing up and booking an appointment, a doctor approving an appointment request, a patient viewing medication information, and an admin changing a user's role or removing an error log entry.

1.3 Core Features

- **Authentication**: Login, signup, password hashing with bcrypt, JWT access tokens, refresh tokens, logout, and session restoration.
- **Role-Based Routing**: Each role sees only the pages needed for that role. Logged-out users only see support, team info, login, and signup.
- **Patient Dashboard**: Patients can view notifications, maintain health details, and see relevant account information.
- **Appointment System**: Patients request appointments. Doctors manage availability and approve or reject appointment requests.
- **Doctor Portal**: Doctors view appointment queues, patient data, availability slots, and prescription-related data.
- **Pharmacy**: Patients view generic medications and medications assigned to them. Patients can also use a mock cart and checkout.
- **Admin Portal**: Admins create, read, update, and delete user accounts. Admins can also review and remove error logs.
- **Error Logger**: Frontend and backend errors are saved into MongoDB in an `error_log` collection and displayed inside the admin portal.
- **AI Assistant**: A Gemini-based assistant appears as a logged-in-only chat widget. It uses current user context from MongoDB and applies guardrails for health-related answers.
- **Support and Team Info**: Public pages provide project, team, and support information.

1.4 Originality vs Inspiration

The project is original in implementation and was designed specifically for the class final project. The general idea is inspired by real hospital management portals and patient-facing healthcare dashboards. Our version differs from a production hospital system by focusing on a course-appropriate scope: a clear role-based UI, basic CRUD operations, MongoDB-backed persistence, and a lightweight AI enhancement rather than full clinical, billing, or insurance functionality.

2 System Overview and Project Description

2.1 Major Functional Modules

- **Authentication Module**: Implemented in `backend/api/auth.js` and `frontend/src/pages/Login`. It handles signup, login, JWT access tokens, refresh tokens, logout, and session verification.
- **Routing and Role Module**: Implemented mainly in `frontend/src/App.jsx` and `frontend/src/components/Navbar.jsx`. It maps browser paths to React views and redirects users away from pages outside their role.
- **Admin Module**: Implemented in `backend/api/admin.js` and `frontend/src/pages/Admin.jsx`. It manages users and error logs.
- **Appointments Module**: Implemented in `backend/api/appointments.js` and `frontend/src/page`. It manages doctors, availability slots, patient appointment requests, and doctor appointment decisions.
- **Doctor Portal Module**: Implemented in `frontend/src/pages/DoctorPortal.jsx`. It gives doctors a single workspace for appointments, patient data, availability, and prescriptions.
- **Pharmacy Module**: Implemented in `backend/api/pharmacy.js` and `frontend/src/pages/Pharma`. It manages patient medication visibility, assigned medications, prices, stock, and the patient cart.
- **User Settings Module**: Implemented in `backend/api/users.js` and `frontend/src/pages/Securi`. It supports profile, password, and patient health details.
- **Error Logging Module**: Implemented in `backend/api/errors.js`, `backend/config/errorLog.js` and `frontend/src/utils/errorLogger.js`. It captures client and server errors for admin review.
- **AI Assistant Module**: Implemented in `backend/api/assistant.js` and `frontend/src/componen`. It sends authenticated chat questions to Gemini with limited user context.

The frontend communicates with the backend using `fetch`. Protected requests send the JWT access token in the `Authorization: Bearer <token>` header. The backend validates the token using middleware before reading or writing MongoDB data.

2.2 UI and Navigation Flow

The application uses a single-page React structure. `App.jsx` reads the browser path and chooses which page component to render. The navigation bar changes based on the logged-in user's role.

- **Logged-out users**: Can access Support, Team Info, Login, and Sign Up.
- **Patients**: After login, patients are sent to the Dashboard. They can access Dashboard, Appointments, Pharmacy, Settings, Support, and Team Info.
- **Doctors**: After login, doctors are sent to the Doctor Portal. They can access Doctor Portal, Settings, Support, and Team Info.
- **Admins**: After login, admins are sent to Admin. They can access Admin, Settings, Support, and Team Info.

If a user manually enters a route outside their role, the frontend redirects them to the correct default page. For example, an admin does not need pharmacy, so an admin attempting to visit `/pharmacy` is redirected to `/admin`.

2.3 CRUD Entities

- **Users**: Fields include name, email, role, passwordHash, phone, specialization, health details, createdAt, and updatedAt. Admins can create, read, update, and delete users. Passwords are stored as bcrypt hashes.
- **Appointments**: Fields include patientId, doctorId, date, time, type, reason, comments, and status. Patients create appointment requests. Doctors read, approve, reject, and manage appointment data.
- **Doctor Availability**: Fields include doctorId, date, time, active, createdAt, and updatedAt. Doctors create and delete availability slots.
- **Medications**: Fields include name, strength, dosageForm, directions, category, stock, price, isGeneric, active, createdAt, and updatedAt.
- **Medicine Assignments**: Fields include patientId, medicationId, assignedBy, dosage, directions, notes, status, createdAt, and updatedAt. Doctors can assign medications to patients.
- **Notifications**: Fields include userId, message, type, read, and createdAt. The system creates notifications when appointments or prescriptions change.
- **Error Logs**: Fields include message, source, severity, stack, URL, method, status, user information, context, and createdAt. Admins read and delete error log records.
- **Refresh Tokens**: Fields include userId, tokenHash, createdAt, and expiresAt. These are created and deleted by the authentication flow.

3 File and Folder Architecture

3.1 Folder Structure Overview

The project is split into `backend/` and `frontend/`. This separation keeps API logic and UI logic independent. The backend owns authentication, validation, database access, and third-party API calls. The frontend owns page rendering, navigation, form state, and user interactions. This structure also makes the project easier to run because each side has its own `package.json`.

3.2 Backend Folder Structure

3.2.1 Backend Folder Tree

```
backend/
  package.json
  server.js
  api/
    admin.js
    appointments.js
    assistant.js
    auth.js
    errors.js
    notification.js
    pharmacy.js
    users.js
  config/
    db.js
    errorLog.js
    initDb.js
```

3.2.2 Backend Folder Description

- `server.js`: Main Express entry point. It loads environment variables, configures CORS and JSON parsing, mounts API routers, handles 404 responses, and logs server errors.
- `api/auth.js`: Handles login, signup, refresh token rotation, logout, and `requireAuth` middleware.
- `api/admin.js`: Handles admin user management and error log review/removal.

- `api/appointments.js`: Handles doctor availability, patient appointment requests, doctor approval/rejection, and patient/doctor appointment reads.
- `api/pharmacy.js`: Handles medication inventory, patient medication visibility, medicine assignments, and prescription-related routes.
- `api/users.js`: Handles profile, password update, and patient health details.
- `api/errors.js`: Receives frontend error logs.
- `api/assistant.js`: Handles Gemini AI assistant requests using authenticated user context.
- `api/notification.js`: Handles patient notification data.
- `config/db.js`: Creates and reuses the MongoDB Atlas connection.
- `config/initDb.js`: Creates collections, validation rules, indexes, and the default admin account on backend startup.
- `config/errorLog.js`: Provides helper functions for writing to the `error_log` collection.

3.3 Frontend Folder Structure

3.3.1 Frontend Folder Tree

```
frontend/src/
  App.jsx
  index.css
  main.jsx
  components/
    AIAssistant.jsx
    Footer.jsx
    Navbar.jsx
  pages/
    Admin.jsx
    Appointments.jsx
    Dashboard.jsx
    DoctorPortal.jsx
    Home.jsx
    Login.jsx
    NotFound.jsx
    Pharmacy.jsx
    SecuritySettings.jsx
    Support.jsx
```

```
    TeamInfo.jsx
  utils/
    errorLogger.js
```

3.3.2 Frontend Folder Description

- `App.jsx`: Central route controller. It stores authentication state, restores sessions, redirects roles, and renders pages.
- `index.css`: Global CSS for the hospital theme, forms, tables, dashboards, pharmacy, admin tools, and AI assistant widget.
- `components/Navbar.jsx`: Role-based navigation bar.
- `components/Footer.jsx`: Shared footer with team information.
- `components/AIAssistant.jsx`: Floating Gemini assistant chat widget.
- `pages/Login.jsx`: Login/signup page with password visibility toggle.
- `pages/Admin.jsx`: User CRUD and error log management.
- `pages/Dashboard.jsx`: Patient dashboard, notifications, and health details.
- `pages/Appointments.jsx`: Patient appointment booking.
- `pages/DoctorPortal.jsx`: Doctor appointment, availability, patient, and prescription workspace.
- `pages/Pharmacy.jsx`: Patient medication and cart view.
- `pages/SecuritySettings.jsx`: Profile and password settings.
- `pages/Support.jsx` and `pages/TeamInfo.jsx`: Public informational pages.
- `utils/errorLogger.js`: Shared frontend helper for sending errors to the backend.

4 Code Explanation and Logic Flow

4.1 Frontend–Backend Interaction

The frontend and backend work together through API requests. The React frontend is what the user sees in the browser. When the user clicks a button or submits a form, the frontend uses `fetch` to send a request to the Express backend. The backend checks the request, talks to MongoDB if needed, and sends data back to the frontend.

For example, when a user logs in, the login form sends the email and password to `/api/auth/login`. If the login is correct, the backend sends back the user information and an access token. The frontend saves that token and uses it on protected pages like dashboard, appointments, pharmacy, settings, doctor portal, and admin.

Protected requests include this header:

```
Authorization: Bearer <accessToken>
```

The backend checks this token with the `requireAuth` middleware. If the token is valid, the backend knows which user is making the request. This is how the app makes sure patients only see patient pages, doctors only see doctor tools, and admins see admin tools.

The frontend stores returned data in React state. For example, the pharmacy page stores medication data so it can display medication cards and update the cart. If something fails, the page shows an error message and can also send the error to the backend error log.

4.2 React Component Example

The AI assistant is a good example of how one React component works. It only appears after a user logs in. It keeps track of whether the chat box is open, the messages in the conversation, what the user is typing, and whether the assistant is currently loading.

```
function AIAssistant({ accessToken, apiBase }) {
  const [isOpen, setIsOpen] = useState(false);
  const [messages, setMessages] = useState([
    { role: "assistant", text: "Hi, I can answer basic health questions." },
  ]);
  const [draft, setDraft] = useState("");
  const [isSending, setIsSending] = useState(false);

  async function sendMessage(event) {
    event.preventDefault();
    const message = draft.trim();
    if (!message) return;

    const nextMessages = [...messages, { role: "user", text: message }];
    setMessages(nextMessages);
    setDraft("");
    setIsSending(true);

    const response = await fetch(`${apiBase}/api/assistant/chat`, {
      method: "POST",
      headers: {
```

```
        "Content-Type": "application/json",
        Authorization: `Bearer ${accessToken}`,
      },
      body: JSON.stringify({ message, history: nextMessages.slice(-6) }),
    });
  }
}
```

The main idea is that `useState` stores the changing parts of the component. When the user submits a message, the component first adds the user's message to the chat window. Then it sends a `POST` request to `/api/assistant/chat`. The backend uses the logged-in user's token, sends the question to Gemini, and returns an answer. The answer is then added to the chat.

4.3 API Route Examples

4.3.1 POST Example: Login

`POST /api/auth/login` is used when a user logs in. The backend checks that the email and password were entered. Then it finds the user in MongoDB and checks the password against the saved hash. If the password is correct, the user is logged in and sent to the correct page for their role.

4.3.2 GET Example: Patient Medications

`GET /api/pharmacy/medications` loads the medicine list. For patients, it returns generic medicine plus medicine assigned to that patient. This route is important because it changes what data is shown based on the user's role.

4.3.3 PUT Example: Update Medication

`PUT /api/pharmacy/medications/:id` updates a medicine item. This is restricted to users who are allowed to manage medicine data. The backend checks the medicine ID and the updated fields, then saves the changes in MongoDB.

4.3.4 DELETE Example: Remove Medication

`DELETE /api/pharmacy/medications/:id` removes a medicine from normal use. The app does not fully erase it right away. Instead, it marks it as inactive. This keeps old data available while hiding the medicine from regular patient views.

4.3.5 POST Example: AI Assistant

`POST /api/assistant/chat` sends the user's chat message to the backend. The backend checks that the user is logged in, adds limited user context, and calls Gemini. The Gemini API key stays on the backend, so it is not exposed in the browser.

4.4 Database Interaction

The database is MongoDB Atlas. The backend uses the official MongoDB driver to read and write data. The main thing I understand is that each feature has its own collection or related collections. Users are stored in one collection, appointments in another, medicine in another, and so on.

- **users**: Stores account information, roles, profile details, and patient health details.
- **refreshTokens**: Stores login refresh tokens so users can stay signed in.
- **appointments**: Stores patient appointment requests and doctor decisions.
- **doctorAvailability**: Stores open time slots created by doctors.
- **medications**: Stores pharmacy medicine information like name, strength, stock, price, and whether it is generic.
- **medicineAssignments**: Connects a patient to medicine assigned by a doctor.
- **notifications**: Stores messages shown on the patient dashboard.
- **error_log**: Stores errors so admins can review problems in the system.

Most backend routes use basic database actions like finding one item, finding many items, inserting a new item, updating an item, or deleting/marking something inactive. This is how the app keeps the UI connected to real saved data instead of only temporary frontend data.

5 Screenshots and Web Views

Include screenshots of each important page in the application. For each screenshot, this section briefly describes the purpose of the page, identifies the important UI elements, explains what actions the user can take, describes what happens after interaction, and notes dynamic behavior such as role-based navigation or live data updates.



Figure 1: Home page for Saint Cy's Hospital.

5.1 Home Page

The home page introduces the project as a full-stack hospital platform. The main UI elements shown are the top navigation bar, the Saint Cy's Hospital heading, the project description, and buttons for Log In and Support. A logged-out user can navigate to Support, Team Info, Log In, or Sign Up. Clicking Log In moves the user to the login page, while Support opens the public support page. This page does not submit data to the backend, but it changes navigation based on whether the user is logged in.

5.2 Login and Signup Page

The login page lets users access their account or create a patient account. The key UI elements are the Log In and Sign Up tabs, email input, password input, Show password button, and Log In button. When the user submits the login form, the frontend sends a `POST` request to `/api/auth/login`. If the login works, the app stores the token and sends the user to the correct page for their role. If it fails, an error message is shown. The page also has dynamic tab behavior because the form changes when switching between login and signup.

5.3 Patient Dashboard

The patient dashboard is the main page for a patient after logging in. It shows the patient's name and email, the next appointment card, active prescriptions card, notifications, health

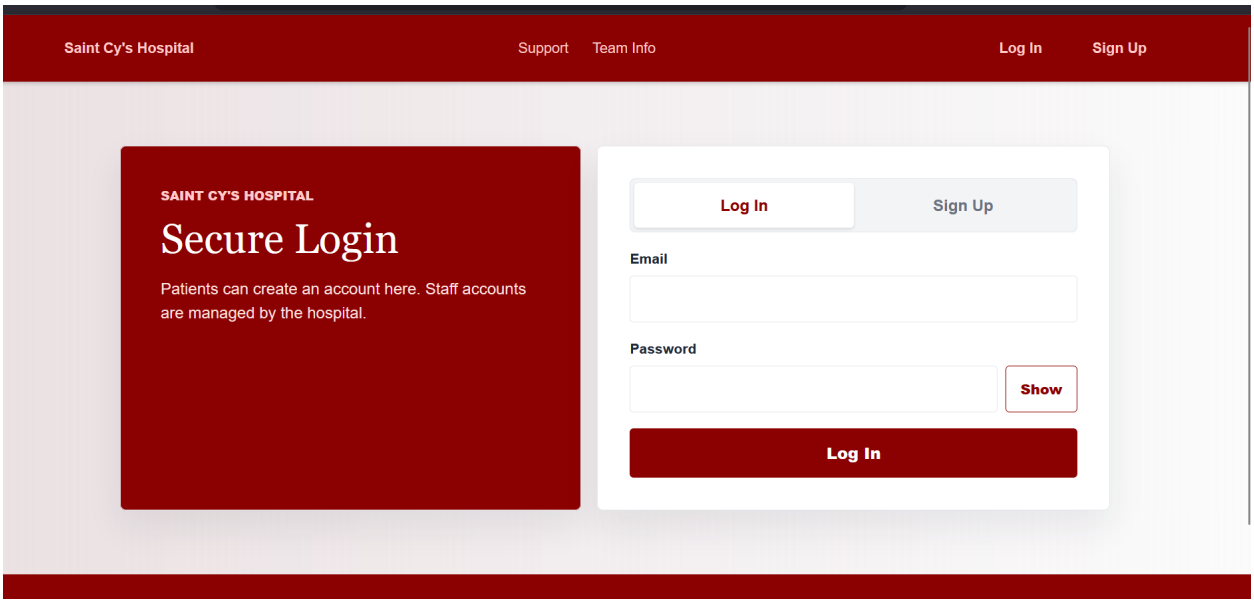


Figure 2: Login and signup page with authentication form.

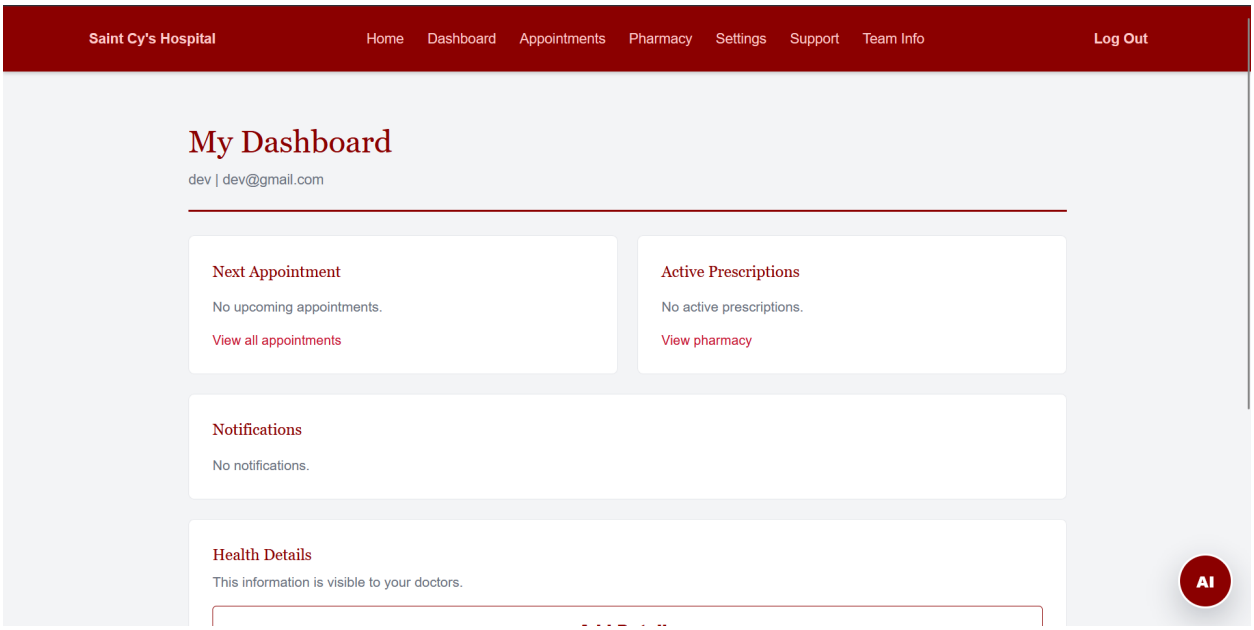


Figure 3: Patient dashboard after login.

details, and the AI assistant button. The user can click View all appointments to go to the appointment page or View pharmacy to open the pharmacy page. Health details can be updated from this area when the full form is visible. The dashboard loads information from backend routes for notifications, appointment data, and user details, then updates the cards based on the returned data.

5.4 Patient Pharmacy View

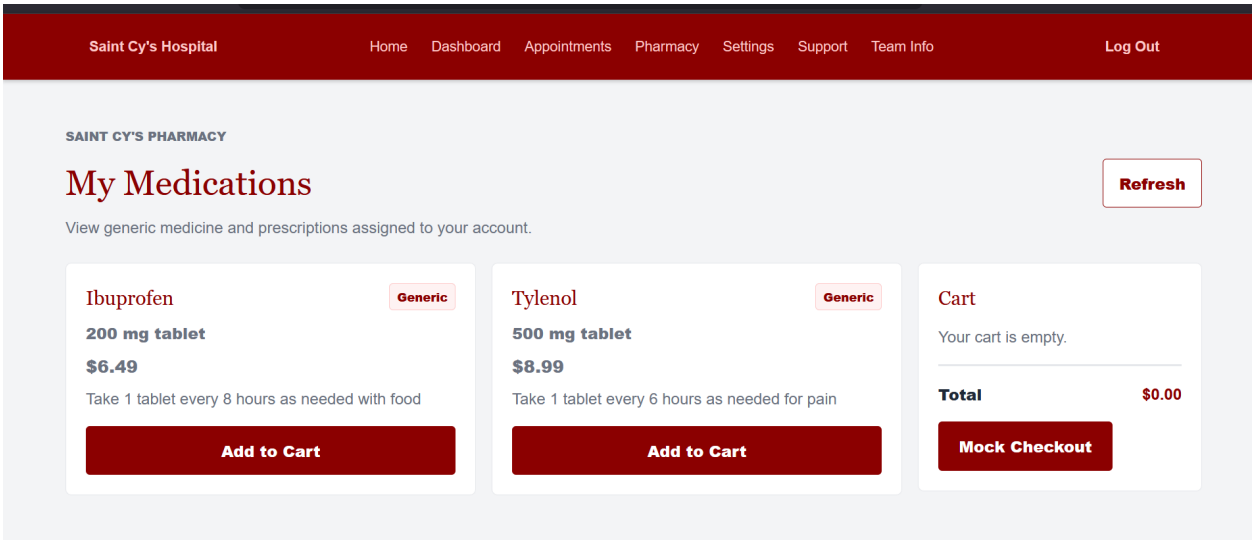


Figure 4: Patient pharmacy page showing available medicine and cart.

The patient pharmacy page lets patients view generic medicine and medicine assigned to their account. The screenshot shows medication cards for Ibuprofen and Tylenol, prices, dosage directions, Add to Cart buttons, a cart panel, total price, Mock Checkout button, and Refresh button. Clicking Add to Cart updates the cart and total on the page. Clicking Refresh reloads medication data from `/api/pharmacy/medications`. Mock Checkout clears or processes the cart on the frontend as a simulated purchase. The dynamic behavior is that the cart total changes when medicine is added or quantity changes.

5.5 Appointments Page

The appointments page lets patients choose a doctor, select an available time, and send an appointment request. The main UI elements are the doctor list, the Book Appointment panel, and the My Appointments section. When the patient clicks a doctor, available time slots are loaded and the booking form becomes useful. Submitting the appointment request sends a `POST` request to an appointments API route, then the request appears in the patient's appointment list. The page updates dynamically because the booking options depend on which doctor is selected.

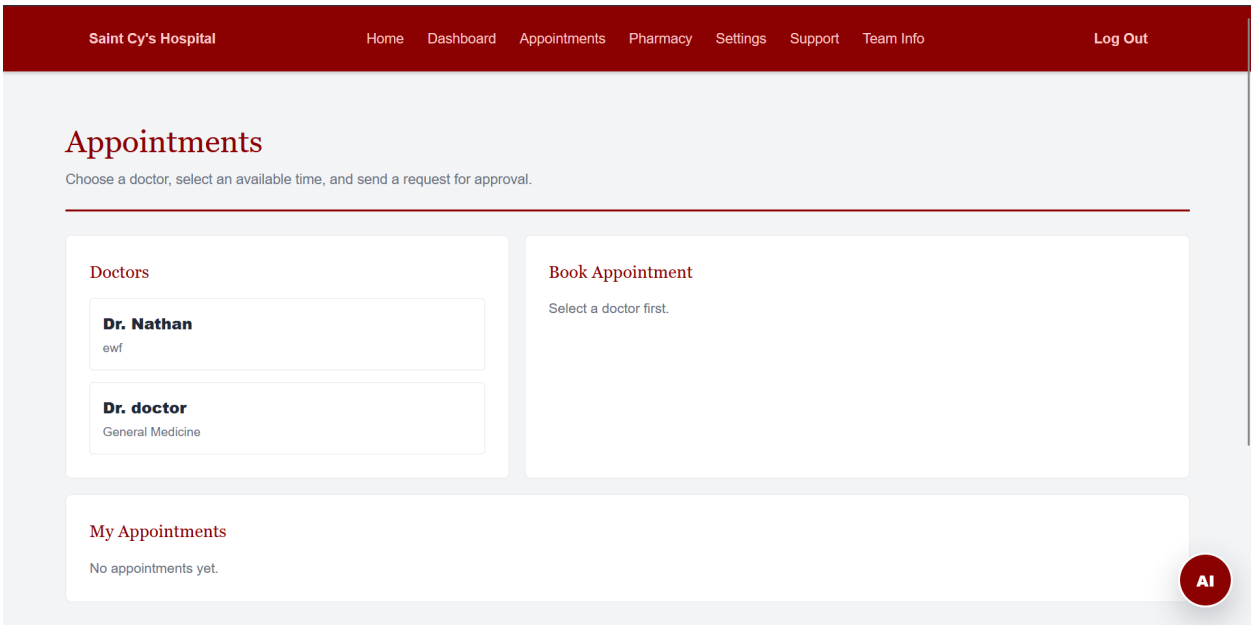


Figure 5: Patient appointment page.

5.6 Doctor Portal

The doctor portal is where doctors manage their work. The screenshot shows the doctor name, pending appointment requests panel, Add Availability form, date input, time input, Add Slot button, Create Prescription section, and AI assistant button. A doctor can approve or reject appointment requests when they exist, add available time slots, view patient-related information, and create prescriptions. Adding availability sends a `POST` request to the appointments availability route and then the page reloads or updates the available slots. The pending request list is dynamic because it changes when patients submit requests or when the doctor approves/rejects them.

5.7 Security and Settings Page

The settings page lets a logged-in user manage account details. In the screenshot, the doctor profile area shows phone and specialization, and the password card includes an Update Password button. The user can update profile information or open the password update form. Updating the profile sends a request to the user/profile API route, and updating the password sends a request to the password API route. After a successful update, the page displays the new saved information or a success message. The form behavior is dynamic because different fields are shown depending on the logged-in user's role.

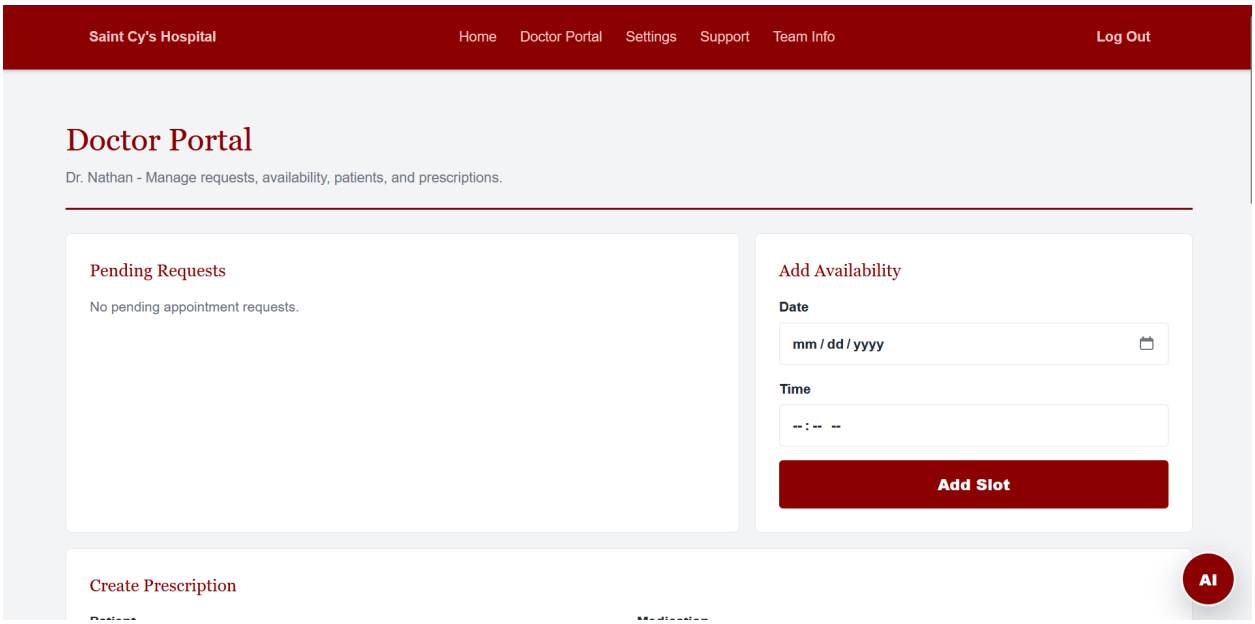


Figure 6: Doctor portal for requests, availability, patients, and prescriptions.

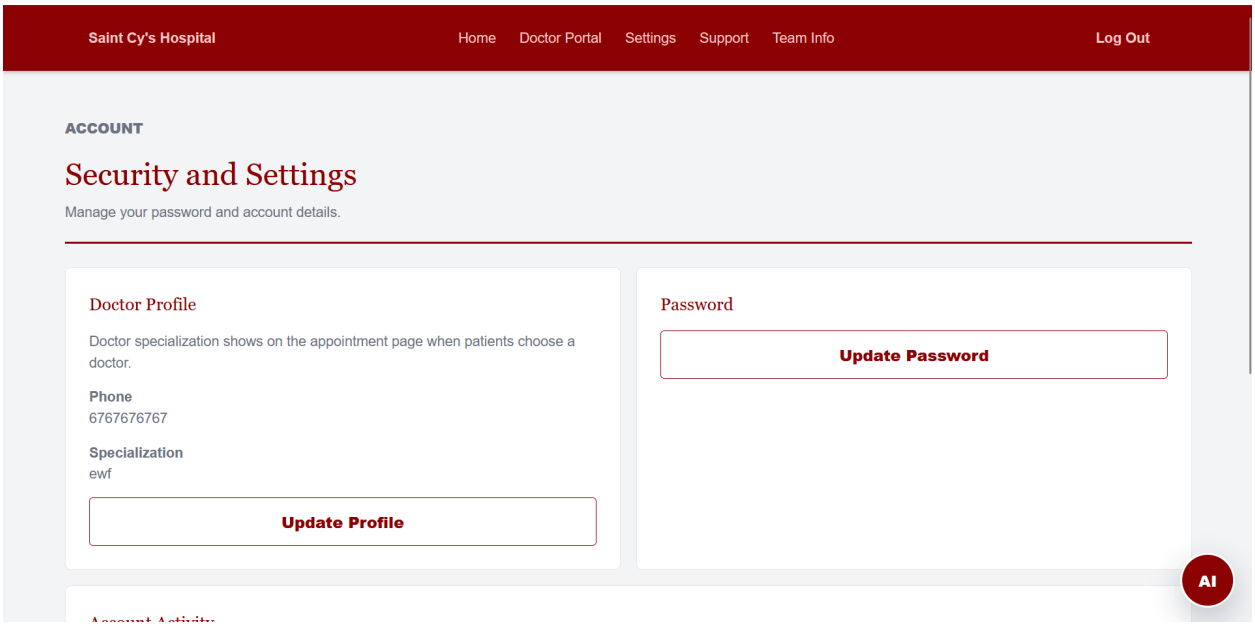


Figure 7: Security and settings page for profile and password changes.

6 Installation and Setup Instructions

6.1 Frontend Setup

- Required runtime: Node.js 18 or newer is recommended.
- Open a terminal in `final_project/ah-17/frontend`.
- Install dependencies:

```
npm install
```

- Start the Vite development server:

```
npm run dev
```

- The frontend usually runs on `http://localhost:5173`.
- Optional frontend environment variable: `VITE_API_URL=http://localhost:5050`.

6.2 Backend Setup

- Required runtime: Node.js 18 or newer is recommended.
- Open a terminal in `final_project/ah-17/backend`.
- Install dependencies:

```
npm install
```

- Start the backend server:

```
node server.js
```

- The backend runs on `http://localhost:5050`.
- On startup, `initDb.js` creates required MongoDB collections, validators, indexes, and a default admin user if no admin exists.

6.3 Database Setup

The project uses MongoDB Atlas. Create an Atlas cluster and a database named `saint_cys`. The backend creates the main collections automatically when `MONGODB_URI` and `MONGODB_DB` are configured.

Required collections are:

- `users`
- `refreshTokens`
- `appointments`
- `doctorAvailability`
- `medications`
- `medicineAssignments`
- `notifications`
- `error_log`

Seed data can be inserted manually through MongoDB Atlas. For example, medication documents need fields such as `name`, `strength`, `dosageForm`, `stock`, `price`, `isGeneric`, and `active`. Generic medications are visible to all patients, while non-generic medications are visible only when assigned.

6.4 Environment Variables

- `PORT`: Backend port, default `5050`.
- `CLIENT_ORIGIN`: Allowed frontend origins for CORS.
- `MONGODB_URI`: MongoDB Atlas connection string.
- `MONGODB_DB`: Database name, usually `saint_cys`.
- `USERS_COLLECTION`: User collection name, usually `users`.
- `REFRESH_TOKENS_COLLECTION`: Refresh token collection name.
- `JWT_SECRET`: Secret used to sign JWT access tokens.
- `JWT_EXPIRES_IN`: Access token lifetime, such as `15m`.
- `REFRESH_TOKEN_DAYS`: Refresh token lifetime in days.

- `DEFAULT_ADMIN_NAME`: Name for the initial admin account.
- `DEFAULT_ADMIN_EMAIL`: Email for the initial admin account.
- `DEFAULT_ADMIN_PASSWORD`: Password for the initial admin account.
- `GEMINI_API_KEY`: API key for the Gemini AI assistant.
- `GEMINI_MODEL`: Gemini model name, such as `gemini-2.5-flash`.

7 Contribution Overview

7.1 Devank Uppal Contributions

- Built the authentication system including login, signup, bcrypt password hashing, JWT access tokens, refresh tokens, session restore, and logout.
- Implemented role-based frontend routing in `App.jsx` and role-based navigation in `Navbar.jsx`.
- Built the admin portal for user CRUD, role editing, password resets, self-delete protection, and delete confirmation.
- Implemented the error logging system with frontend logging utilities, backend error storage, and admin error log display/removal.
- Built the pharmacy backend routes and frontend pharmacy page, including medication CRUD, patient assignment rules, prices, stock, generic medication visibility, patient cart, quantity controls, and mock checkout.
- Added MongoDB initialization logic for users, refresh tokens, error logs, medications, and medicine assignments.
- Integrated the Gemini AI assistant with backend guardrails, logged-in-only frontend widget, and MongoDB-backed user context.
- Added and debugged route restrictions so each role only sees and accesses relevant pages.
- Debugged backend restart issues where old Node processes caused route-not-found errors.

7.2 Neal Sharma Contributions

- Built patient dashboard functionality for notifications and health details.
- Built the appointment workflow including doctor selection, date/time slot selection, appointment request creation, and patient appointment list.
- Built doctor portal workflows for reviewing appointment requests, managing availability, viewing patients, and working with prescriptions.
- Built account settings/profile functionality for phone, specialization, password updates, and patient health details.
- Added support and team information pages to provide public project information.
- Worked on frontend styling and page layout for dashboard, appointments, doctor portal, settings, support, and team views.
- Helped connect frontend pages to backend routes using authenticated API calls.

8 Challenges Faced

8.1 Devank Uppal Challenges

- **Authentication and sessions**: The challenge was keeping users logged in without storing plain passwords or long-lived access tokens. The solution was bcrypt for password hashes, short JWT access tokens, and hashed refresh tokens stored in MongoDB.
- **Role-based routing**: The challenge was preventing users from seeing pages that did not match their role. The solution was a central `canAccessRoute` function and role-specific navbar rendering.
- **Backend stale process bugs**: Several route-not-found errors happened because an older Node process was still running on port 5050. This was diagnosed by checking direct API responses and port ownership, then fixed by stopping the old process and restarting the backend.
- **Pharmacy visibility rules**: The challenge was making generic medicine public to all patients while keeping assigned-only medicine private. The solution was a backend query that returns `isGeneric: true` medications plus active assignments for the current patient.
- **AI guardrails**: The challenge was adding an AI feature without exposing private data or giving unsafe medical advice. The solution was to keep the Gemini key server-side, send only current-user context, check emergency terms, and instruct Gemini to provide general guidance only.

8.2 Neal Sharma Challenges

- **Appointment workflow**: The challenge was connecting doctors, time slots, and patient requests in a way that stayed understandable in the UI. The solution was to separate doctor availability from appointment requests and use clear status values.
- **Doctor portal data**: The challenge was showing doctors only the patients and appointments relevant to them. The solution was to filter appointment and patient data by the logged-in doctor ID.
- **Patient dashboard**: The challenge was keeping patient information readable while still supporting editable health data. The solution was a dashboard layout with separate cards for notifications, health details, and related account information.
- **Settings page**: The challenge was managing profile and password update forms without mixing responsibilities. The solution was to separate profile updates from password updates and use individual save states.

9 Final Reflections

9.1 Devank Uppal Reflection

This project taught me a lot about what it is actually like to connect a React frontend to a real backend instead of only building screens with fake data. My biggest focus was authentication, role-based routing, the admin tools, pharmacy features, error logging, and the AI assistant. At first, some parts felt straightforward in isolation, but once users, roles, MongoDB data, and API routes all had to work together, the project became more challenging.

One thing I learned was how important it is to test the backend directly and not only rely on what the browser shows. There were times when a bug looked like a frontend issue, but the real problem was an old backend process still running or an API route not being hit correctly. I also learned more about protecting data by keeping JWT checks on protected routes and making sure users only saw pages that matched their role.

If I could restart the project, I would plan the database structure earlier. Some features, especially pharmacy prices, assignments, and visibility rules, would have been easier if the schema had been finalized before the UI was built. Overall, this project made me more comfortable working across the full stack and helped me understand how much planning matters when multiple features share the same users and data.

9.2 Neal Sharma Reflection

This project helped me get better at building React pages that are connected to actual backend data. I worked mostly on the patient dashboard, appointment workflow, doctor

portal, settings, support, and team pages. The most useful part for me was seeing how user actions on the frontend, like booking an appointment or updating profile information, had to connect cleanly to API calls and database updates.

The appointment system was one of the harder parts because it involved both patients and doctors. Patients needed a simple way to request appointments, while doctors needed a different view for availability and appointment decisions. Building those two sides helped me understand why role-based workflows need to be designed carefully instead of just making one general page for everyone.

If I could do the project again, I would spend more time planning the data passed between the frontend and backend before starting the UI. Some changes took longer because the page layout and API responses had to be adjusted at the same time. Overall, I learned more about React state, forms, authenticated requests, and organizing a project so that each role has a clear purpose in the application.